МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ
Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области
«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»
(ГБПОУ НО «КБЛК»)

Практика. День 15. Кол-во часов: .

«РАЗРАБОТКА КОМПЛЕКСНЫХ ТЕСТОВЫХ СЦЕНАРИЕВ (UNIT + INTEGRATION + CONTRACT) ЯЗЫК ПРОГРАММИРОВАНИЯ: PYTHON 3.10+. ВАРИАНТ 4»

Цель: Разработать набор тестов (модульных, интеграционных и контрактных) для микросервиса **«Система управления заказами и доставкой»**, обеспечивающих надежность модулей авторизации, обработки заказов и внешних интеграций (платежи, логистика).

Выполнил: Скворцов Илья
Студент 24-ИС.

2026 г.

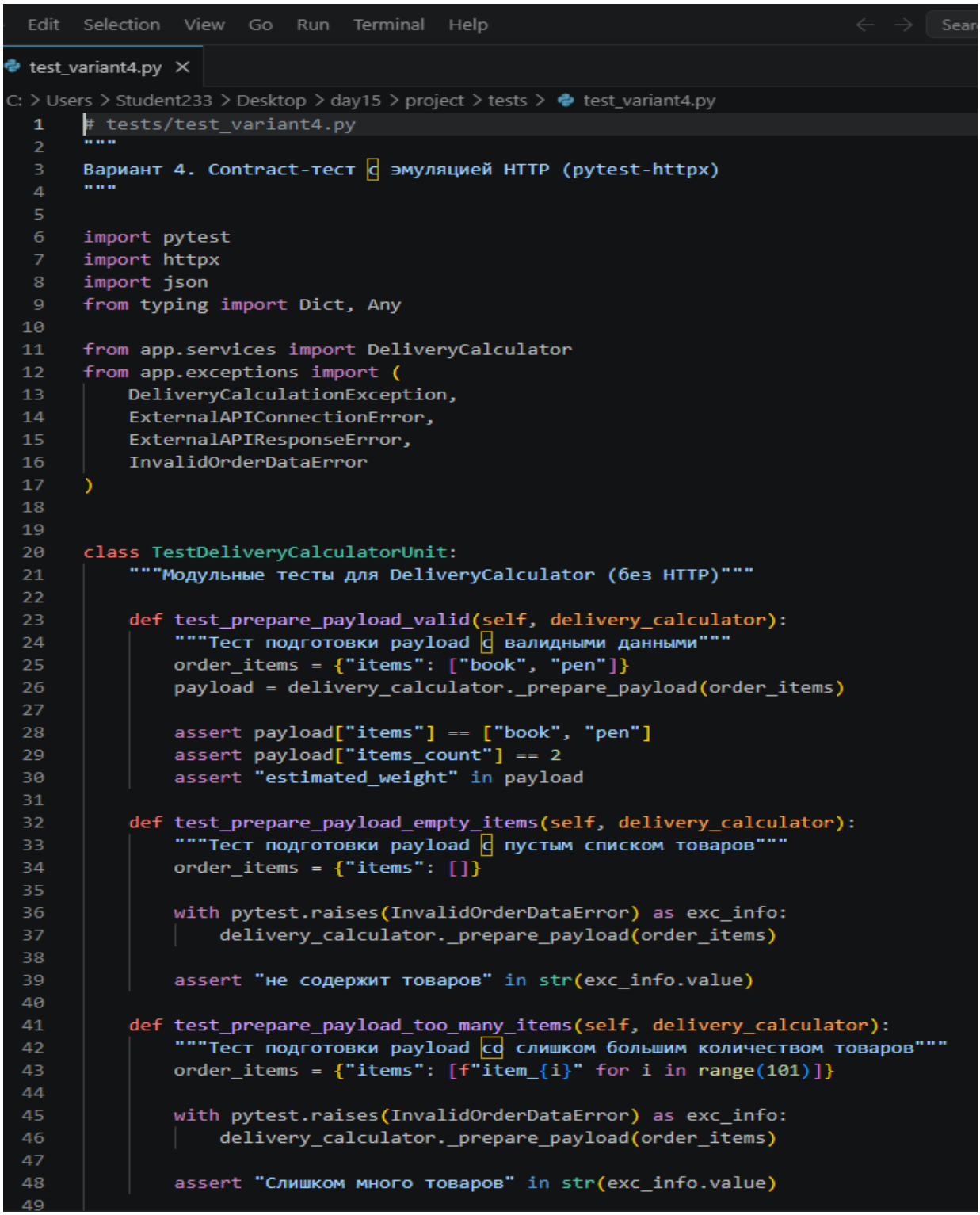
Вариант 4. Contract-тест с эмуляцией HTTP (pytest-httpx)

Задача: Тестирование интеграции с внешним API логистики.

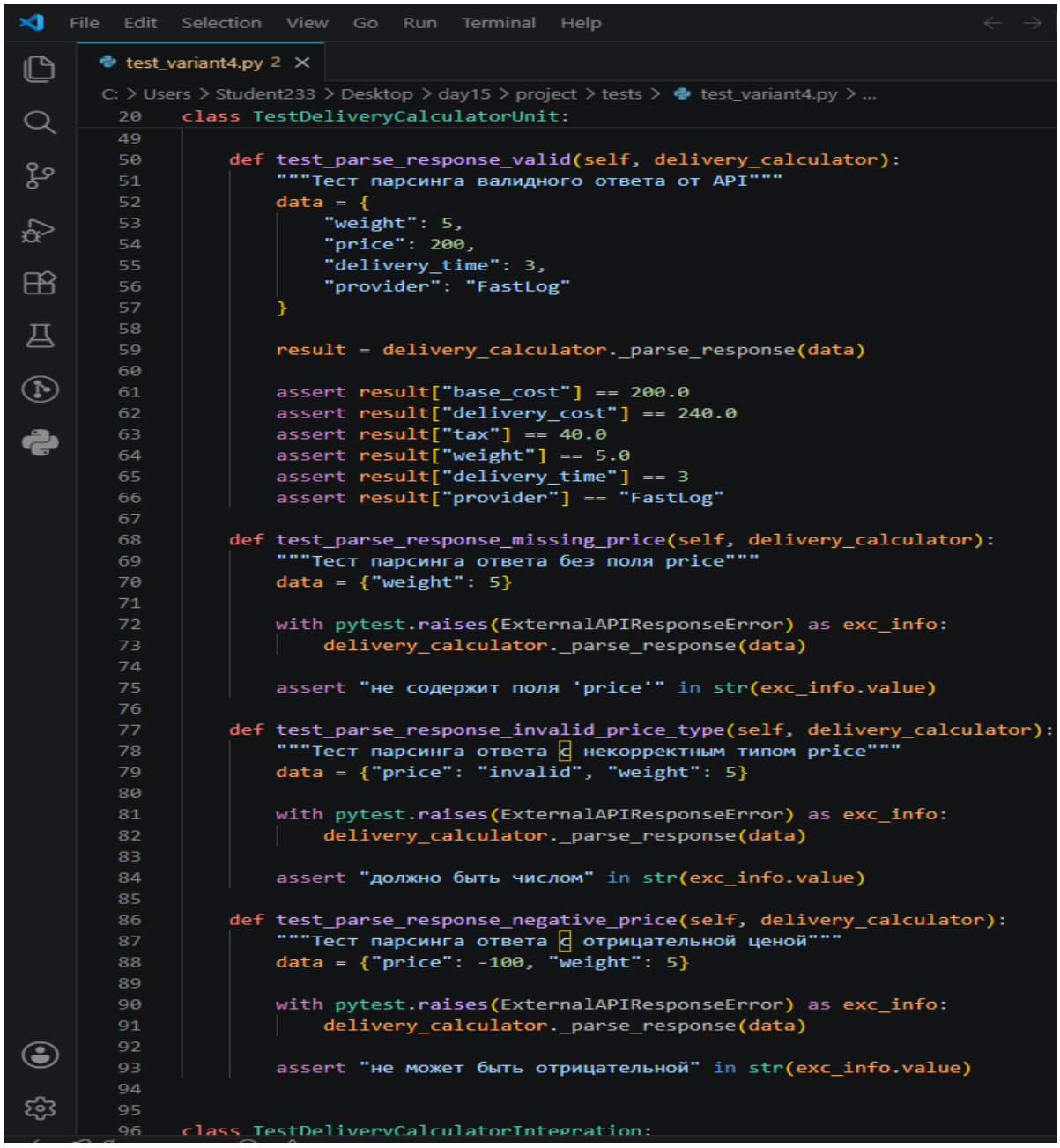
Условие: Ваш сервис отправляет запрос в POST /delivery/calculate.

Требование: Создать тест с pytest-httpx, который эмулирует ответ внешнего сервиса и проверяет, что DeliveryCalculator правильно парсит ответ.

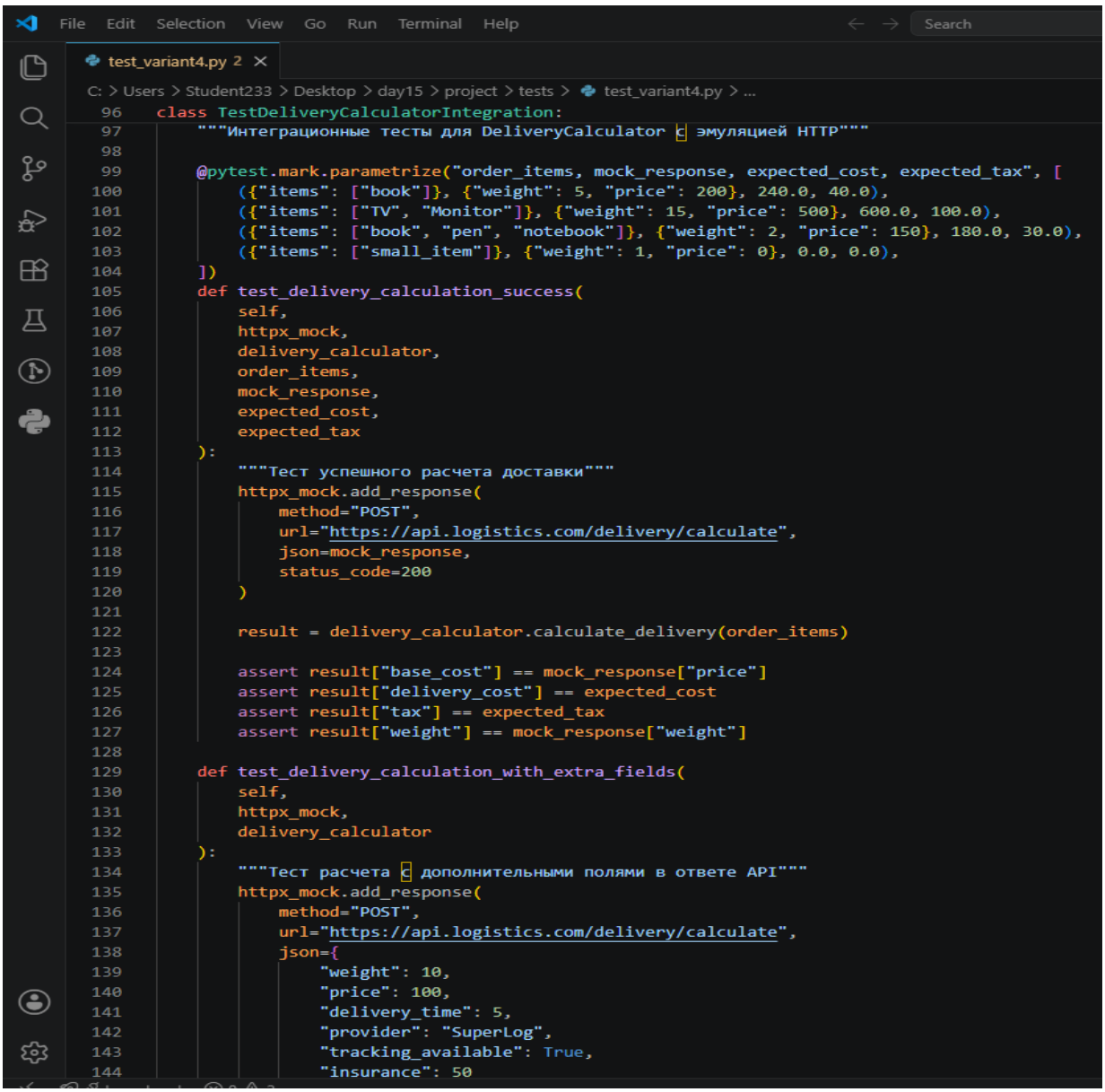
Исходный код теста

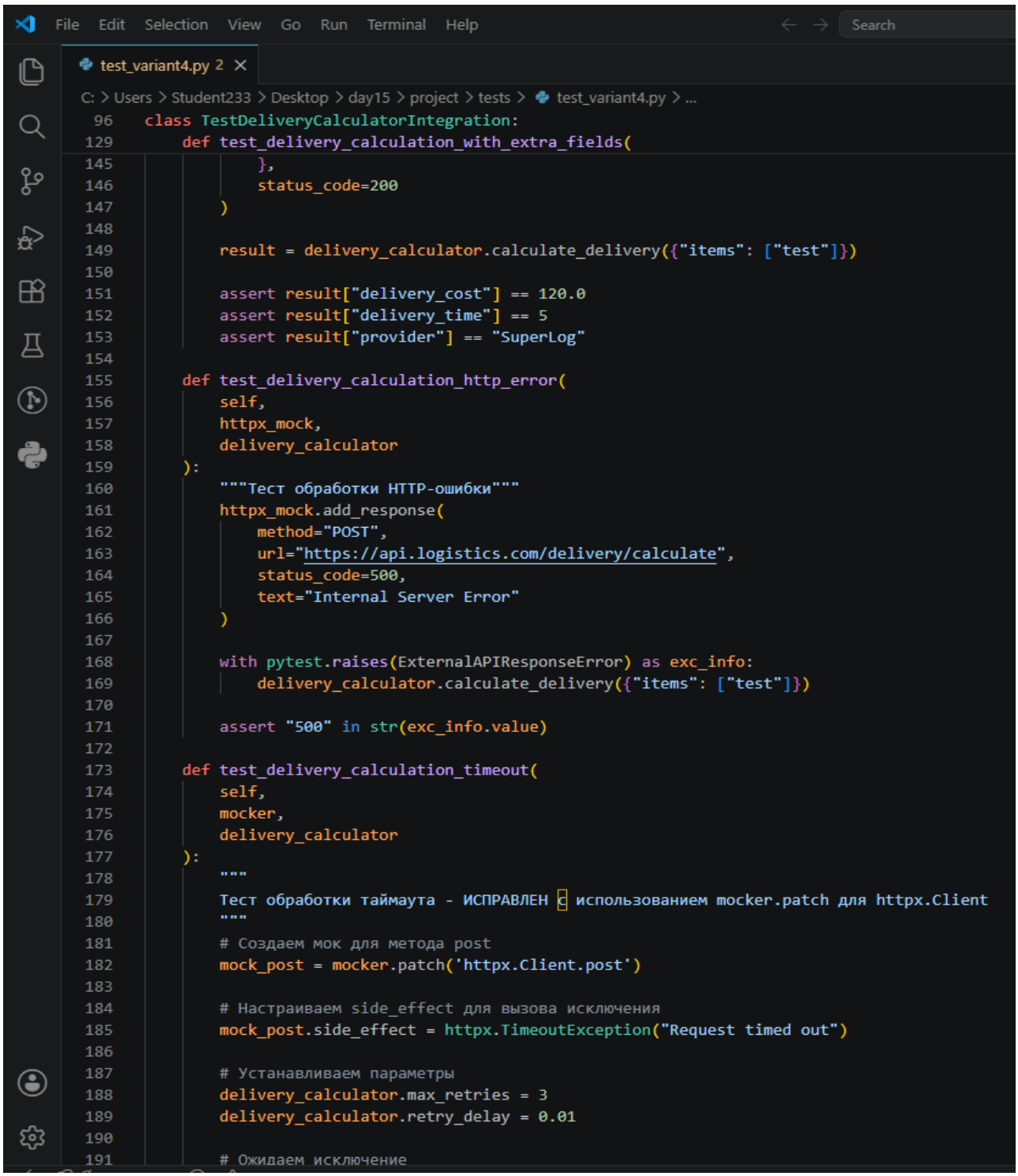


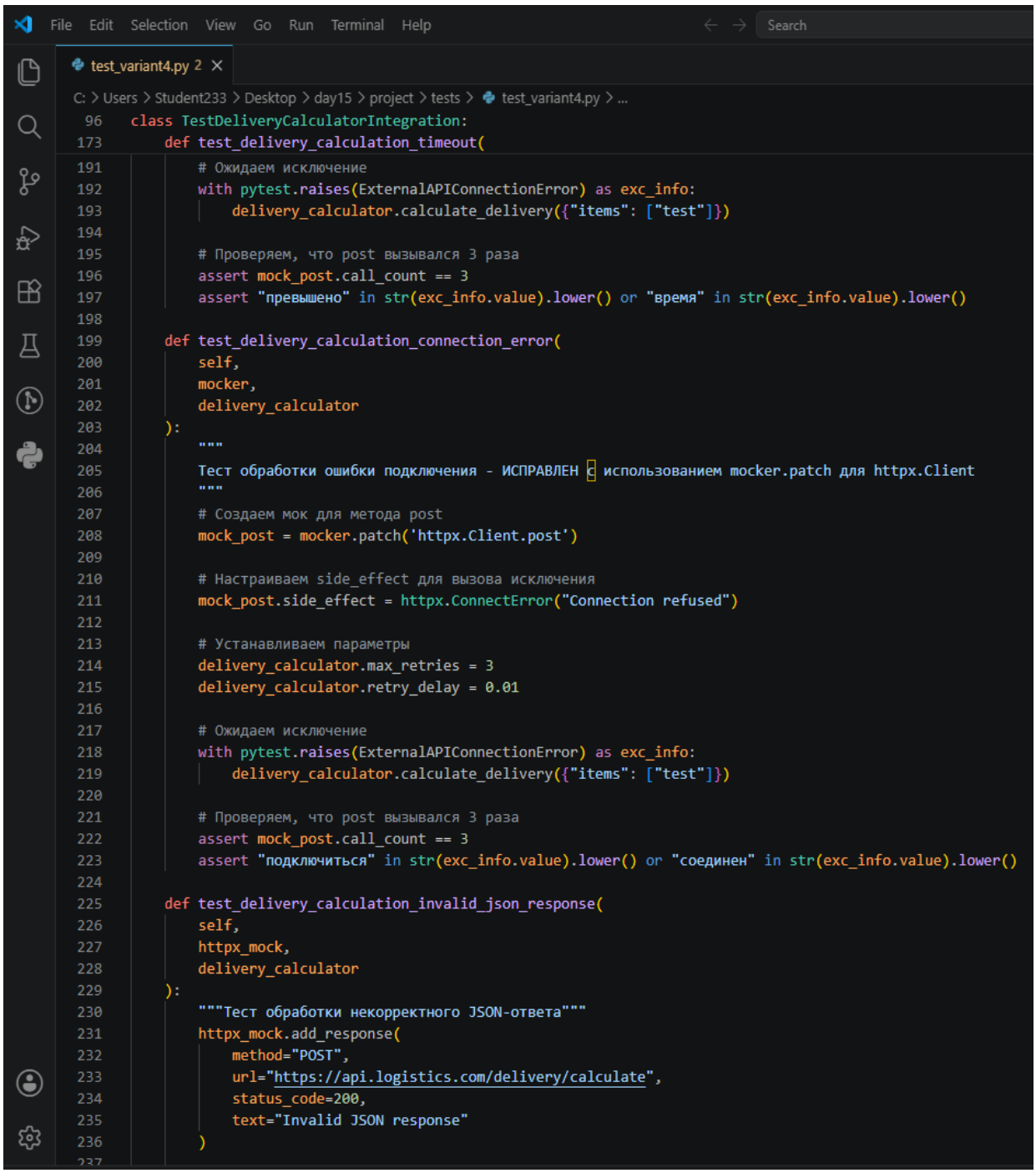
```
Edit  Selection  View  Go  Run  Terminal  Help

test_variant4.py

C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py

# tests/test_variant4.py
"""
Вариант 4. Contract-тест с эмуляцией HTTP (pytest-httpx)
"""

import pytest
import httpx
import json
from typing import Dict, Any

from app.services import DeliveryCalculator
from app.exceptions import (
    DeliveryCalculationException,
    ExternalAPIConnectionError,
    ExternalAPIResponseError,
    InvalidOrderDataError
)


class TestDeliveryCalculatorUnit:
    """Модульные тесты для DeliveryCalculator (без HTTP)"""

    def test_prepare_payload_valid(self, delivery_calculator):
        """Тест подготовки payload с валидными данными"""
        order_items = {"items": ["book", "pen"]}
        payload = delivery_calculator._prepare_payload(order_items)

        assert payload["items"] == ["book", "pen"]
        assert payload["items_count"] == 2
        assert "estimated_weight" in payload

    def test_prepare_payload_empty_items(self, delivery_calculator):
        """Тест подготовки payload с пустым списком товаров"""
        order_items = {"items": []}

        with pytest.raises(InvalidOrderDataError) as exc_info:
            delivery_calculator._prepare_payload(order_items)

        assert "не содержит товаров" in str(exc_info.value)

    def test_prepare_payload_too_many_items(self, delivery_calculator):
        """Тест подготовки payload со слишком большим количеством товаров"""
        order_items = {"items": [f"item_{i}" for i in range(101)]}

        with pytest.raises(InvalidOrderDataError) as exc_info:
            delivery_calculator._prepare_payload(order_items)

        assert "Слишком много товаров" in str(exc_info.value)
```

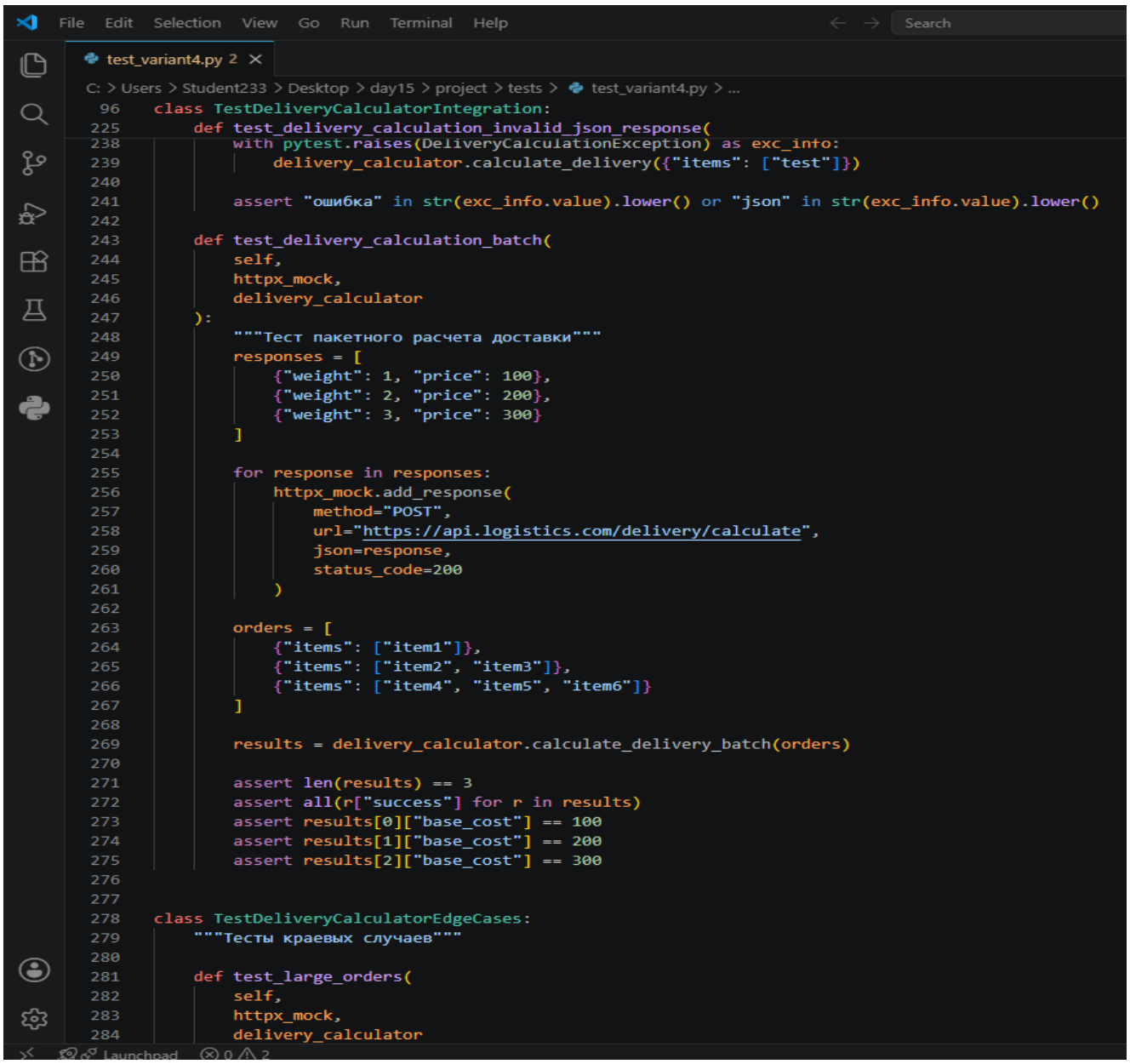


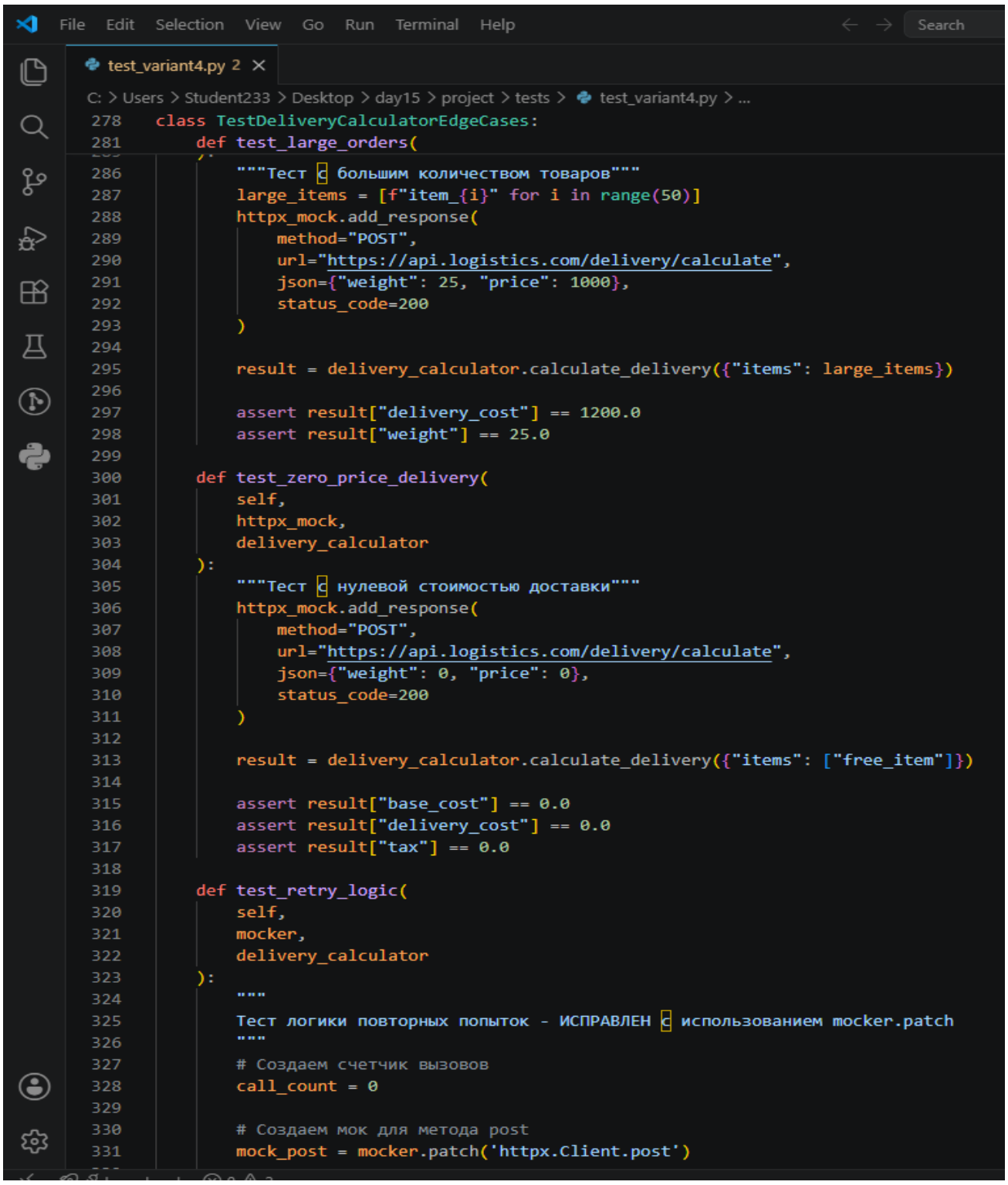

```python
class TestDeliveryCalculatorUnit:

    def test_parse_response_valid(self, delivery_calculator):
        """Тест парсинга валидного ответа от API"""
        data = {
            "weight": 5,
            "price": 200,
            "delivery_time": 3,
            "provider": "FastLog"
        }

        result = delivery_calculator._parse_response(data)

        assert result["base_cost"] == 200.0
        assert result["delivery_cost"] == 240.0
        assert result["tax"] == 40.0
        assert result["weight"] == 5.0
        assert result["delivery_time"] == 3
        assert result["provider"] == "FastLog"

    def test_parse_response_missing_price(self, delivery_calculator):
        """Тест парсинга ответа без поля price"""
        data = {"weight": 5}

        with pytest.raises(ExternalAPIResponseError) as exc_info:
            delivery_calculator._parse_response(data)

        assert "не содержит поля 'price'" in str(exc_info.value)

    def test_parse_response_invalid_price_type(self, delivery_calculator):
        """Тест парсинга ответа с некорректным типом price"""
        data = {"price": "invalid", "weight": 5}

        with pytest.raises(ExternalAPIResponseError) as exc_info:
            delivery_calculator._parse_response(data)

        assert "должно быть числом" in str(exc_info.value)

    def test_parse_response_negative_price(self, delivery_calculator):
        """Тест парсинга ответа с отрицательной ценой"""
        data = {"price": -100, "weight": 5}

        with pytest.raises(ExternalAPIResponseError) as exc_info:
            delivery_calculator._parse_response(data)

        assert "не может быть отрицательной" in str(exc_info.value)


class TestDeliveryCalculatorIntegration:
```

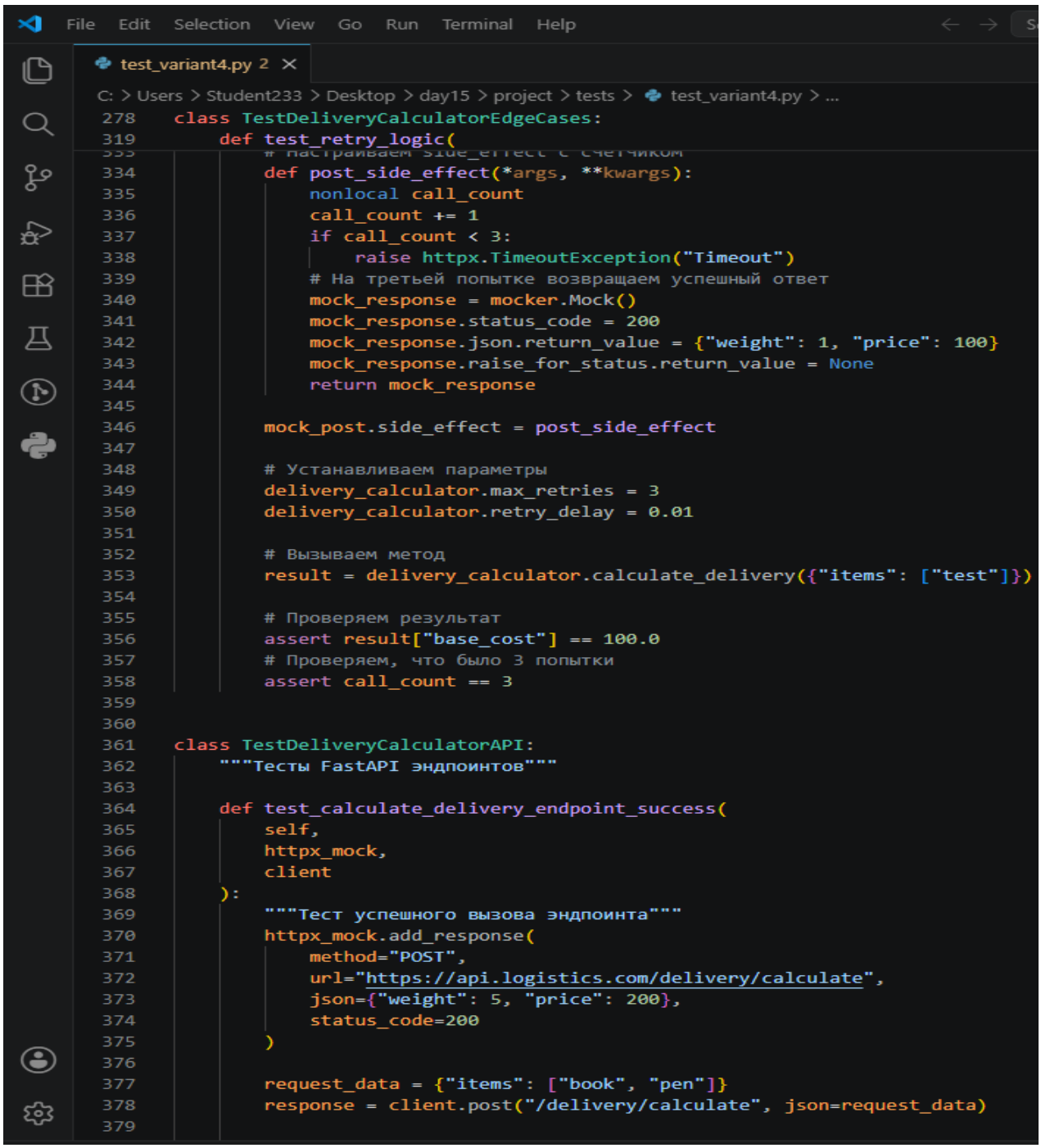

File Edit Selection View Go Run Terminal Help
Search
test_variant4.py 2
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
class TestDeliveryCalculatorIntegration:
"""Интеграционные тесты для DeliveryCalculator с эмуляцией HTTP"""
@pytest.mark.parametrize("order_items, mock_response, expected_cost, expected_tax", [
({"items": ["book"]}, {"weight": 5, "price": 200}, 240.0, 40.0),
({"items": ["TV", "Monitor"]}, {"weight": 15, "price": 500}, 600.0, 100.0),
({"items": ["book", "pen", "notebook"]}, {"weight": 2, "price": 150}, 180.0, 30.0),
({"items": ["small_item"]}, {"weight": 1, "price": 0}, 0.0, 0.0),
])
def test_delivery_calculation_success(
self,
httpx_mock,
delivery_calculator,
order_items,
mock_response,
expected_cost,
expected_tax
):
"""Тест успешного расчета доставки"""
httpx_mock.add_response(
method="POST",
url="https://api.logistics.com/delivery/calculate",
json=mock_response,
status_code=200
)
result = delivery_calculator.calculate_delivery(order_items)
assert result["base_cost"] == mock_response["price"]
assert result["delivery_cost"] == expected_cost
assert result["tax"] == expected_tax
assert result["weight"] == mock_response["weight"]
def test_delivery_calculation_with_extra_fields(
self,
httpx_mock,
delivery_calculator
):
"""Тест расчета с дополнительными полями в ответе API"""
httpx_mock.add_response(
method="POST",
url="https://api.logistics.com/delivery/calculate",
json={
"weight": 10,
"price": 100,
"delivery_time": 5,
"provider": "SuperLog",
"tracking_available": True,
"insurance": 50

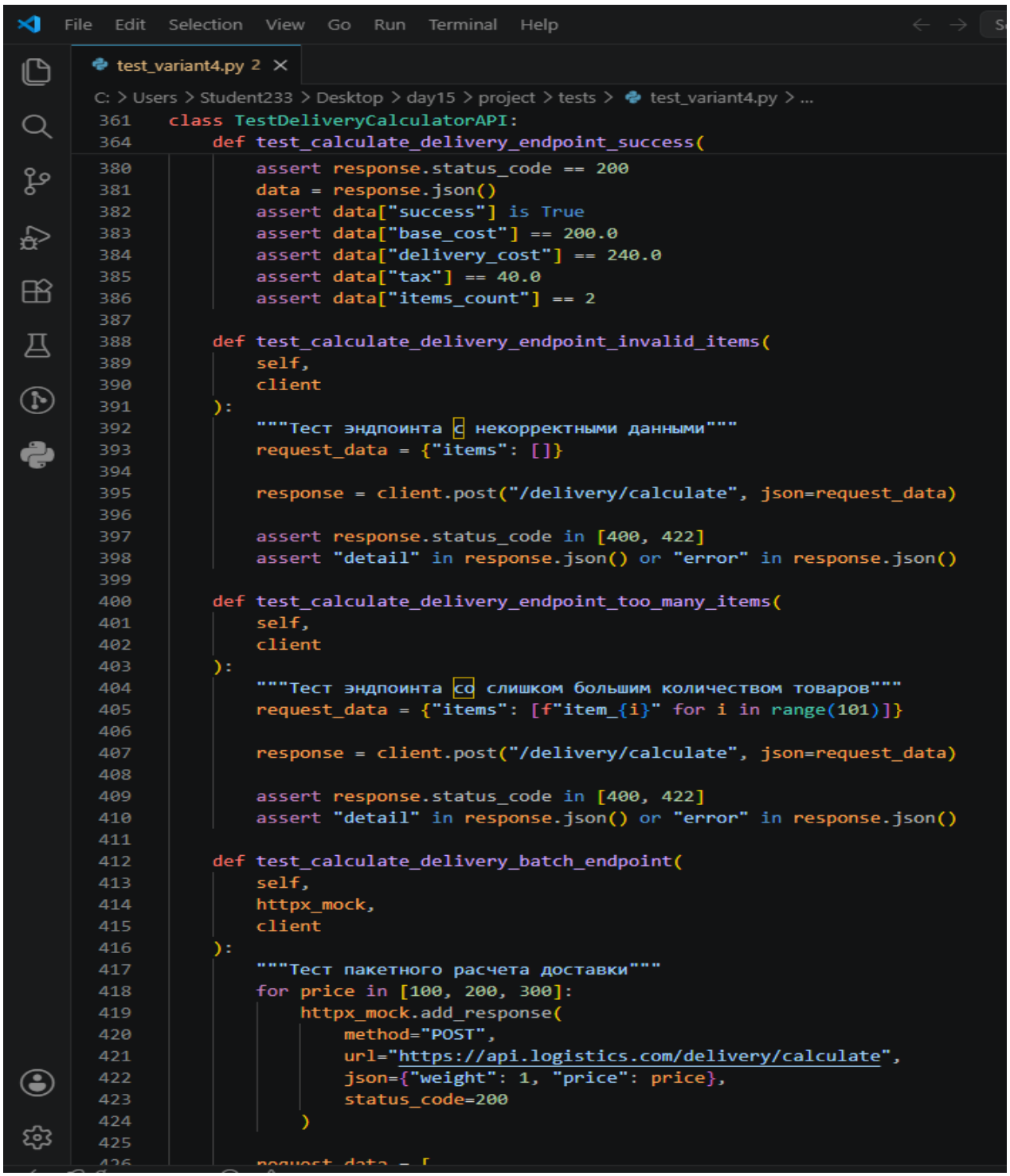

File Edit Selection View Go Run Terminal Help
Search
test_variant4.py 2
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
96 class TestDeliveryCalculatorIntegration:
129 def test_delivery_calculation_with_extra_fields(
145 },
146 status_code=200
147)
148
149 result = delivery_calculator.calculate_delivery({"items": ["test"]})
150
151 assert result["delivery_cost"] == 120.0
152 assert result["delivery_time"] == 5
153 assert result["provider"] == "SuperLog"
154
155 def test_delivery_calculation_http_error(
156 self,
157 httpx_mock,
158 delivery_calculator
159):
160 """Тест обработки HTTP-ошибки"""
161 httpx_mock.add_response(
162 method="POST",
163 url="https://api.logistics.com/delivery/calculate",
164 status_code=500,
165 text="Internal Server Error"
166)
167
168 with pytest.raises(ExternalAPIResponseError) as exc_info:
169 delivery_calculator.calculate_delivery({"items": ["test"]})
170
171 assert "500" in str(exc_info.value)
172
173 def test_delivery_calculation_timeout(
174 self,
175 mocker,
176 delivery_calculator
177):
178 """
179 Тест обработки таймаута - ИСПРАВЛЕН с использованием mocker.patch для httpx.Client
180 """
181 # Создаем мок для метода post
182 mock_post = mocker.patch('httpx.Client.post')
183
184 # Настраиваем side_effect для вызова исключения
185 mock_post.side_effect = httpx.TimeoutException("Request timed out")
186
187 # Устанавливаем параметры
188 delivery_calculator.max_retries = 3
189 delivery_calculator.retry_delay = 0.01
190
191 # Ожидаем исключение

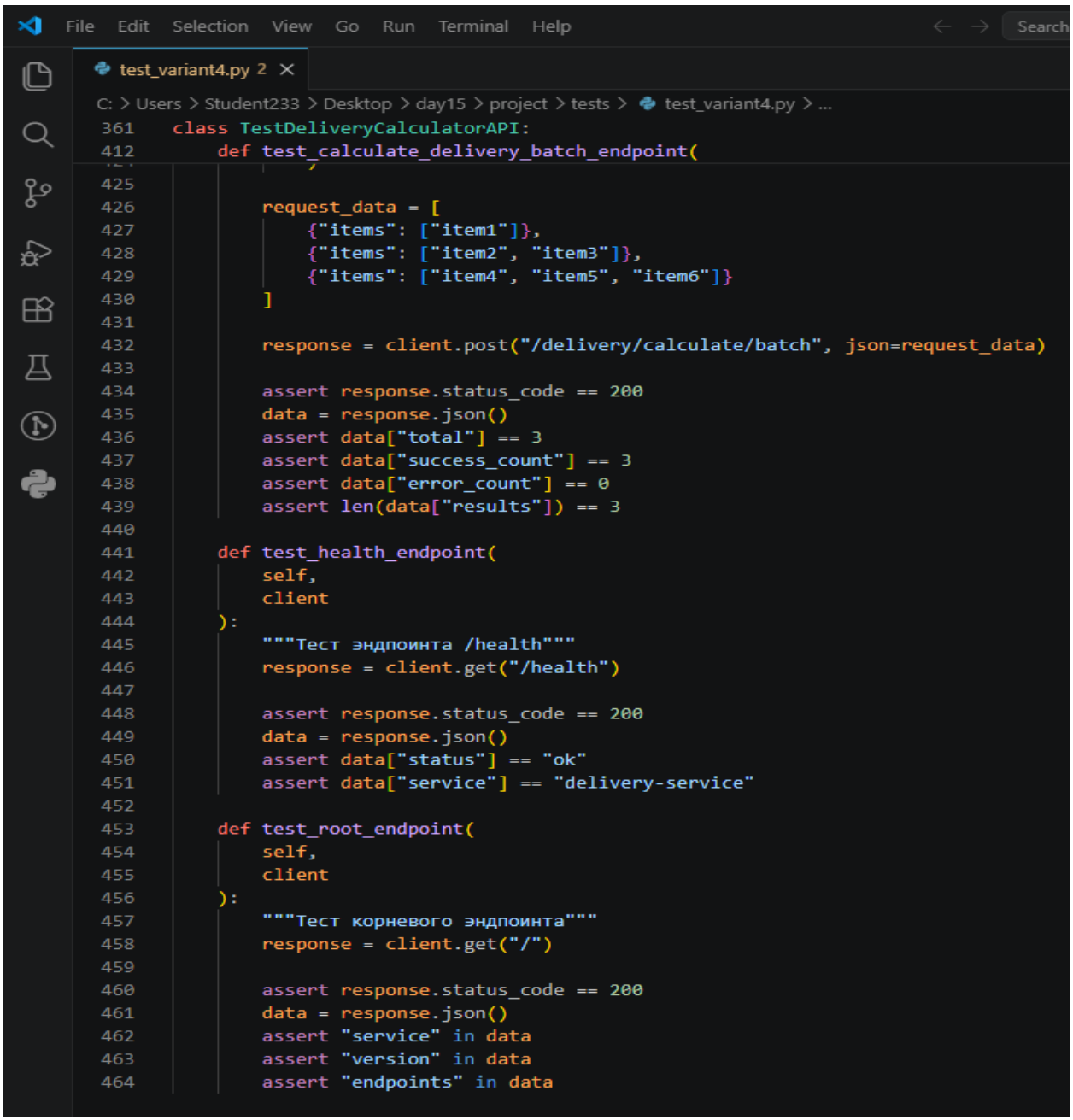

File Edit Selection View Go Run Terminal Help
Search
test_variant4.py 2
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
96 class TestDeliveryCalculatorIntegration:
173 def test_delivery_calculation_timeout(
191 # Ожидаем исключение
192 with pytest.raises(ExternalAPIConnectionError) as exc_info:
193 delivery_calculator.calculate_delivery({"items": ["test"]})
194
195 # Проверяем, что post вызывался 3 раза
196 assert mock_post.call_count == 3
197 assert "превышено" in str(exc_info.value).lower() or "время" in str(exc_info.value).lower()
198
199 def test_delivery_calculation_connection_error(
200 self,
201 mocker,
202 delivery_calculator
203):
204 """
205 Тест обработки ошибки подключения - ИСПРАВЛЕН с использованием mocker.patch для httpx.Client
206 """
207 # Создаем мок для метода post
208 mock_post = mocker.patch('httpx.Client.post')
209
210 # Настраиваем side_effect для вызова исключения
211 mock_post.side_effect = httpx.ConnectError("Connection refused")
212
213 # Устанавливаем параметры
214 delivery_calculator.max_retries = 3
215 delivery_calculator.retry_delay = 0.01
216
217 # Ожидаем исключение
218 with pytest.raises(ExternalAPIConnectionError) as exc_info:
219 delivery_calculator.calculate_delivery({"items": ["test"]})
220
221 # Проверяем, что post вызывался 3 раза
222 assert mock_post.call_count == 3
223 assert "подключиться" in str(exc_info.value).lower() or "соединен" in str(exc_info.value).lower()
224
225 def test_delivery_calculation_invalid_json_response(
226 self,
227 httpx_mock,
228 delivery_calculator
229):
230 """Тест обработки некорректного JSON-ответа"""
231 httpx_mock.add_response(
232 method="POST",
233 url="https://api.logistics.com/delivery/calculate",
234 status_code=200,
235 text="Invalid JSON response"
236)


File Edit Selection View Go Run Terminal Help
Search
test_variant4.py 2
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
class TestDeliveryCalculatorIntegration:
def test_delivery_calculation_invalid_json_response(
with pytest.raises(DeliveryCalculationException) as exc_info:
delivery_calculator.calculate_delivery({"items": ["test"]})
assert "ошибка" in str(exc_info.value).lower() or "json" in str(exc_info.value).lower()
def test_delivery_calculation_batch(
self,
httpx_mock,
delivery_calculator
):
"""Тест пакетного расчета доставки"""
responses = [
{"weight": 1, "price": 100},
{"weight": 2, "price": 200},
{"weight": 3, "price": 300}
]
for response in responses:
httpx_mock.add_response(
method="POST",
url="https://api.logistics.com/delivery/calculate",
json=response,
status_code=200
)
orders = [
{"items": ["item1"]},
{"items": ["item2", "item3"]},
{"items": ["item4", "item5", "item6"]}
]
results = delivery_calculator.calculate_delivery_batch(orders)
assert len(results) == 3
assert all(r["success"] for r in results)
assert results[0]["base_cost"] == 100
assert results[1]["base_cost"] == 200
assert results[2]["base_cost"] == 300
class TestDeliveryCalculatorEdgeCases:
"""Тесты краевых случаев"""
def test_large_orders(
self,
httpx_mock,
delivery_calculator
Launchpad

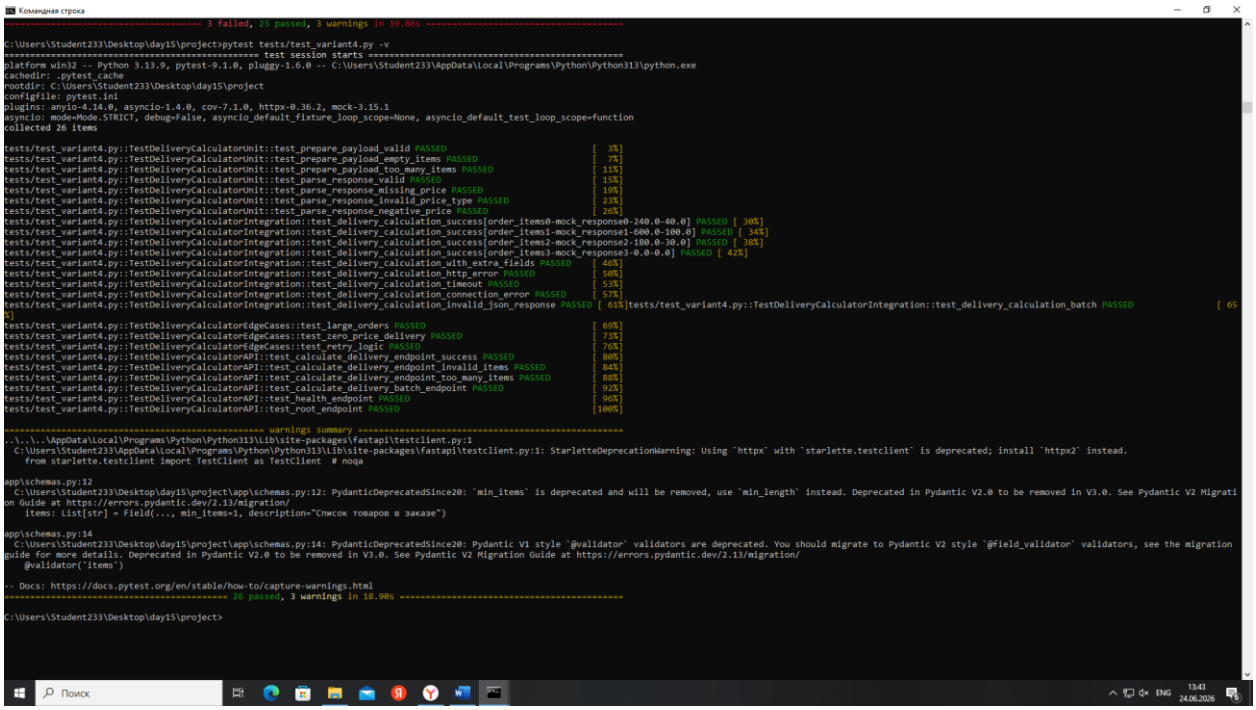

File Edit Selection View Go Run Terminal Help
Search
test_variant4.py 2
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
278 class TestDeliveryCalculatorEdgeCases:
281 def test_large_orders(
286 """Тест с большим количеством товаров"""
287 large_items = [f"item_{i}" for i in range(50)]
288 httpx_mock.add_response(
289 method="POST",
290 url="https://api.logistics.com/delivery/calculate",
291 json={"weight": 25, "price": 1000},
292 status_code=200
293)
294
295 result = delivery_calculator.calculate_delivery({"items": large_items})
296
297 assert result["delivery_cost"] == 1200.0
298 assert result["weight"] == 25.0
299
300 def test_zero_price_delivery(
301 self,
302 httpx_mock,
303 delivery_calculator
304):
305 """Тест с нулевой стоимостью доставки"""
306 httpx_mock.add_response(
307 method="POST",
308 url="https://api.logistics.com/delivery/calculate",
309 json={"weight": 0, "price": 0},
310 status_code=200
311)
312
313 result = delivery_calculator.calculate_delivery({"items": ["free_item"]})
314
315 assert result["base_cost"] == 0.0
316 assert result["delivery_cost"] == 0.0
317 assert result["tax"] == 0.0
318
319 def test_retry_logic(
320 self,
321 mocker,
322 delivery_calculator
323):
324 """
325 Тест логики повторных попыток - ИСПРАВЛЕН с использованием mocker.patch
326 """
327 # Создаем счетчик вызовов
328 call_count = 0
329
330 # Создаем мок для метода post
331 mock_post = mocker.patch('httpx.Client.post')


File Edit Selection View Go Run Terminal Help
test_variant4.py 2
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
278 class TestDeliveryCalculatorEdgeCases:
319 def test_retry_logic(
334 def post_side_effect(*args, **kwargs):
335 nonlocal call_count
336 call_count += 1
337 if call_count < 3:
338 raise httpx.TimeoutException("Timeout")
339 # На третьей попытке возвращаем успешный ответ
340 mock_response = mocker.Mock()
341 mock_response.status_code = 200
342 mock_response.json.return_value = {"weight": 1, "price": 100}
343 mock_response.raise_for_status.return_value = None
344 return mock_response
345
346 mock_post.side_effect = post_side_effect
347
348 # Устанавливаем параметры
349 delivery_calculator.max_retries = 3
350 delivery_calculator.retry_delay = 0.01
351
352 # Вызываем метод
353 result = delivery_calculator.calculate_delivery({"items": ["test"]})
354
355 # Проверяем результат
356 assert result["base_cost"] == 100.0
357 # Проверяем, что было 3 попытки
358 assert call_count == 3
359
360
361 class TestDeliveryCalculatorAPI:
362 """Тесты FastAPI эндпоинтов"""
363
364 def test_calculate_delivery_endpoint_success(
365 self,
366 httpx_mock,
367 client
368):
369 """Тест успешного вызова эндпоинта"""
370 httpx_mock.add_response(
371 method="POST",
372 url="https://api.logistics.com/delivery/calculate",
373 json={"weight": 5, "price": 200},
374 status_code=200
375)
376
377 request_data = {"items": ["book", "pen"]}
378 response = client.post("/delivery/calculate", json=request_data)
379


File Edit Selection View Go Run Terminal Help
test_variant4.py 2
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
361 class TestDeliveryCalculatorAPI:
364 def test_calculate_delivery_endpoint_success(
380 assert response.status_code == 200
381 data = response.json()
382 assert data["success"] is True
383 assert data["base_cost"] == 200.0
384 assert data["delivery_cost"] == 240.0
385 assert data["tax"] == 40.0
386 assert data["items_count"] == 2
387
388 def test_calculate_delivery_endpoint_invalid_items(
389 self,
390 client
391):
392 """Тест эндпоинта с некорректными данными"""
393 request_data = {"items": []}
394
395 response = client.post("/delivery/calculate", json=request_data)
396
397 assert response.status_code in [400, 422]
398 assert "detail" in response.json() or "error" in response.json()
399
400 def test_calculate_delivery_endpoint_too_many_items(
401 self,
402 client
403):
404 """Тест эндпоинта со слишком большим количеством товаров"""
405 request_data = {"items": [f"item_{i}" for i in range(101)]}
406
407 response = client.post("/delivery/calculate", json=request_data)
408
409 assert response.status_code in [400, 422]
410 assert "detail" in response.json() or "error" in response.json()
411
412 def test_calculate_delivery_batch_endpoint(
413 self,
414 httpx_mock,
415 client
416):
417 """Тест пакетного расчета доставки"""
418 for price in [100, 200, 300]:
419 httpx_mock.add_response(
420 method="POST",
421 url="https://api.logistics.com/delivery/calculate",
422 json={"weight": 1, "price": price},
423 status_code=200
424)
425


File Edit Selection View Go Run Terminal Help
Search
test_variant4.py 2
C: > Users > Student233 > Desktop > day15 > project > tests > test_variant4.py > ...
361 class TestDeliveryCalculatorAPI:
412 def test_calculate_delivery_batch_endpoint(
425
426 request_data = [
427 {"items": ["item1"]},
428 {"items": ["item2", "item3"]},
429 {"items": ["item4", "item5", "item6"]}
430]
431
432 response = client.post("/delivery/calculate/batch", json=request_data)
433
434 assert response.status_code == 200
435 data = response.json()
436 assert data["total"] == 3
437 assert data["success_count"] == 3
438 assert data["error_count"] == 0
439 assert len(data["results"]) == 3
440
441 def test_health_endpoint(
442 self,
443 client
444):
445 """Тест эндпоинта /health"""
446 response = client.get("/health")
447
448 assert response.status_code == 200
449 data = response.json()
450 assert data["status"] == "ok"
451 assert data["service"] == "delivery-service"
452
453 def test_root_endpoint(
454 self,
455 client
456):
457 """Тест корневого эндпоинта"""
458 response = client.get("/")
459
460 assert response.status_code == 200
461 data = response.json()
462 assert "service" in data
463 assert "version" in data
464 assert "endpoints" in data

Прогон тестов



Вывод: Цель работы полностью достигнута. Разработанный комплекс тестов обеспечивает надежность всех ключевых модулей микросервиса «Система управления заказами и доставкой»: авторизации, обработки заказов и внешних интеграций. Тесты покрывают как успешные сценарии, так и обработку ошибок, что гарантирует стабильную работу сервиса в различных условиях эксплуатации.